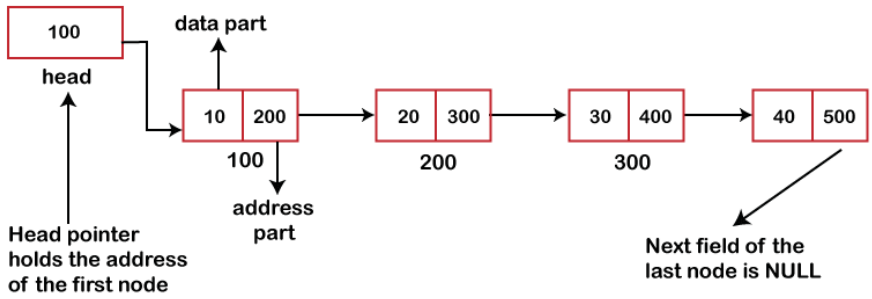
WEEK – 6 SINGLY LINKED LIST

Singly Linked List

Singly linked list is collection of ordered set of elements. A node in singly linked list consists of 2 parts – data part and link(next) part.

Data part stores information while **next** part of node stores address of next node.

It is dynamic data structure i.e., it can be of any length. It can store data of any length.



The last node in the linked list contains **null** as the next pointer to represent the end of the linked list. It can only be traversed in a single direction i.e., can go from first node in a list to last node but cannot move from last node to first node.

Creating the node of Singly Linked List –

1. Create the **node** using the **class**. The class contains **data** and a **pointer** for the **next node**. Create the node with any type of data.

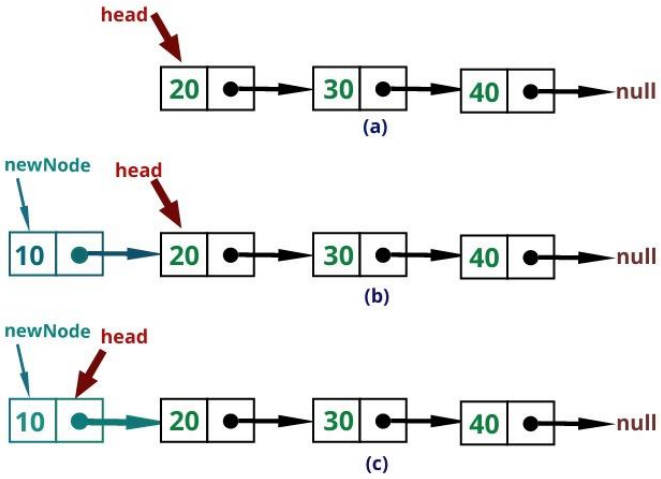
```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

2. Create a class called **LinkedList** with head initialized to **None**.

```
class Linkedlist:
    def __init__(self):
        self.head = None
```

Inserting a new node at the beginning of the linked list -

1. Create a new node with the given data.
2. Make the new node points to the head node.
3. Make the new node as the head node.




```
class Linkedlist:
    def __init__(self):
        self.head = None

    def addbegining(self,data):
        newnode = Node(data)
        newnode.next = self.head
        self.head = newnode
```

Traversing (print) the Nodes

1. Initialize a variable with **head**.
2. Loop that iterates until the end of the linked list is reached.
   - Print the node data.
   - Move the next pointer

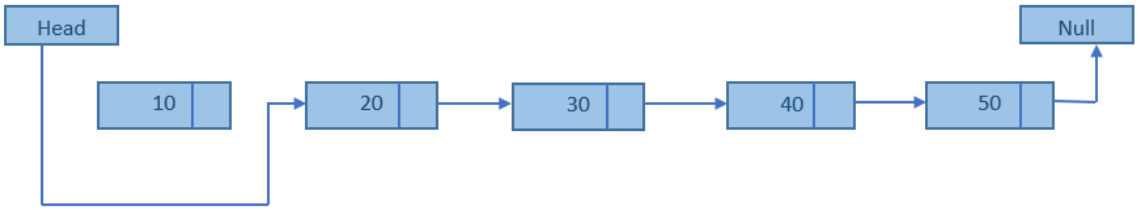
```
  def display(self):
      temp = self.head
      while temp != None:
          print(temp.data, end=' -> ')
          temp = temp.next
      print('Null')
```

Deleting Node at the beginning of linked list

1. Initialize a variable with **head**
2. If linked list is not empty, point head to the second node

```
def DeleteNode(self):
    temp = self.head

    # If Linked list is not empty, point head to second node.
    if (temp != None):
        self.head = temp.next
```



Search a node by value in linked list

1. Make head as the current node.
2. Run a loop until the current node is NULL.
3. In each iteration, check if the key of the node is equal to item. If the key matches the item, return True otherwise return False.

```
def search(self, key):
    temp = self.head
    while temp != None:
        if temp.data == key :
            return True
        else:
            temp = temp.next
    return False
```

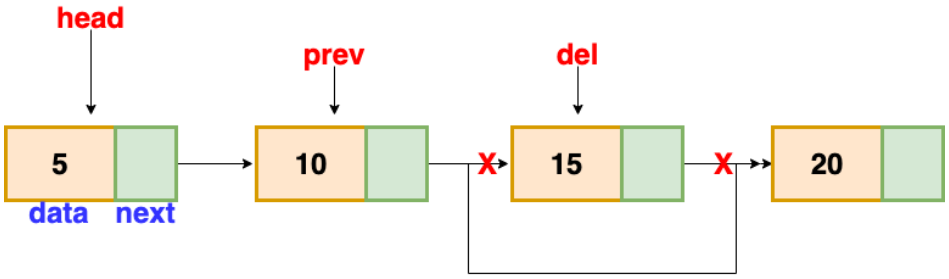
Delete a node by value

To delete a node from the linked list, following are the steps.

1) Find the previous node of the node to be deleted.

2) Change the next of the previous node.

3) Free memory for the node to be deleted.



Deleting a Node in Linked List

```
def deleteNode(self, key):
    # Store head node
    temp = self.head

    # If head node itself holds the key to be deleted
    if (temp is not None):
        if (temp.data == key):
            self.head = temp.next
            temp = None
            return

    # Search for the key to be deleted, keep track of the
    # previous node as we need to change 'prev.next'
    while(temp is not None):
        if temp.data == key:
            break
        prev = temp
        temp = temp.next

    # if key was not present in linked list
    if(temp == None):
        return

# Unlink the node from linked list
prev.next = temp.next
temp = None
```

Time Complexity of Linked List

	Worst case	Average Case
Search	O(n)	O(n)
Insert	O(1)	O(1)
Deletion	O(1)	O(1)

Advantage of Linked List

- Dynamic memory allocation – memory is allocated and deallocated efficiently.
- Easy implementation

Disadvantage of Linked List

- Certain memory is used by pointers.
- Traversal is time consuming as direct access is not possible.

Applications

- Linked list is used in –
- Implementation of stacks and queues
- Implementation of graph
- Performing arithmetic operations on long integers.

Linked List Iterators

Iterators in python is an object that can be iterated upon. An iterator will return data, one element at a time. Built-in containers/collections in python are -list, tuple, string etc.



The built-in function iter( ) takes an iterable object and returns an iterator. Each time when next method is called on the iterator, it gives us next element. If ther are no more elements it raises an stop iteration.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Linkedlist:
    def __init__(self):
        self.head = None

    def AddNode(self,data):
        newnode = Node(data)
        newnode.next = self.head
        self.head = newnode

    def iterate(self):
        temp = self.head
        while(temp != None):
            val = temp.data
            temp = temp.next
            yield val
```

```
# Main
L1 = Linkedlist()
L1.AddNode(5)
L1.AddNode(15)
L1.AddNode("Hello")
L1.AddNode(45)

print("Linked List")
for val in L1.iterate():
    print(val, end="-->")
print("Null")
```

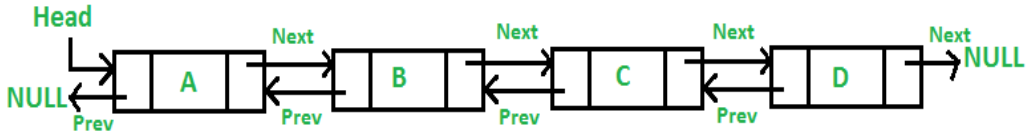
Output:

```
Linked List
45-->Hello-->15-->5-->Null
```

WEEK – 7 DOUBLY LINKED LIST

A doubly linked list is a linear data structure with a collection of sequentially linked nodes. Each node has three parts – two link fields, and one data field.

Data Part: stores the data.
Prev Part: stores the address of the previous node.
Next Part: stores the address of the next node.



Advantages

- Dynamic memory allocation.
- Easy to implement
- Bidirectional traversal

Disadvantages

- Uses extra memory for links per node.
- Elements are to be accessed sequentially due to random storage in the memory.

Applications

- Used by browsers to implement web page navigation
- Used to implement undo/redo operations.
- Used in thread scheduler in operating systems.

Examples 1: Image Viewer
Example 2: Music player list

<u>To create a node in Doubly Linked List</u>

1. Create the **node** using the **class**. The class contains **data** and a **pointer** for the **previous** node and **next node**. Create the node with any type of data.

2. Create a class called **LinkedList** with head initialized to **None**.

```
class Node:
      def __init__(self, data):
            self.data = data
            self.next = None
            self.prev = None

class DoublyLinkedList:
      def __init__(self):
            self.head = None
```

Steps to insert a node at the beginning

The new node is added before the head of the given Linked List. And newly added node becomes the new head of DLL

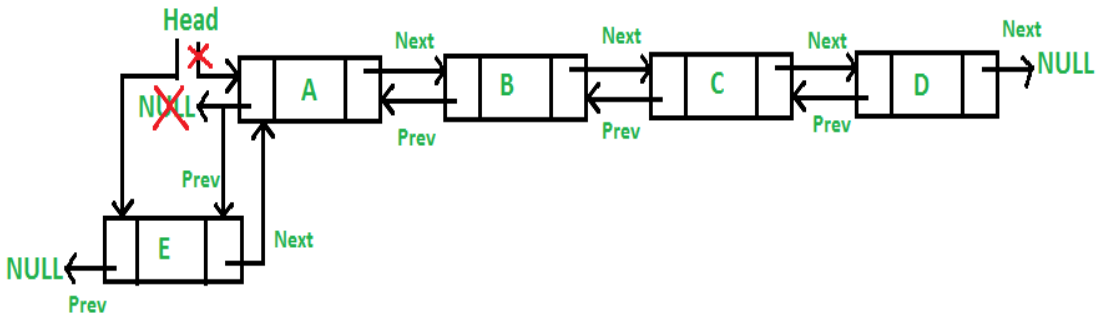
1. **Create the newnode & add data to it.**
   newnode = Node(data)
2. **Change next of newnode as head.**
   newnode.next = self.head

2. **Change prev of head node to newnode if head is not None**. (Linked list not empty)

```
if self.head is not None:
      self.head.prev = newnode
```

4. Move the head to point to the new node

```
self.head = newnode
```



Python Code:

```
#Function to add node in the beginning
def NodeAtBeg(self, data):
    newnode = Node(data)
    newnode.next = self.head
    if self.head != None:
        self.head.prev = newnode
    self.head = newnode
```

To insert a node at the end of the DLL

1. **Create the newnode and add data to it.**

```
new_node = Node(data)
```

2. **If the Linked List is empty, then make the new node as head**

```
if self.head ==  None:
    self.head = new_node
    return
```

3. **Traverse to the last node in doubly linked list.**

```
last = newnode

while last.next != None:

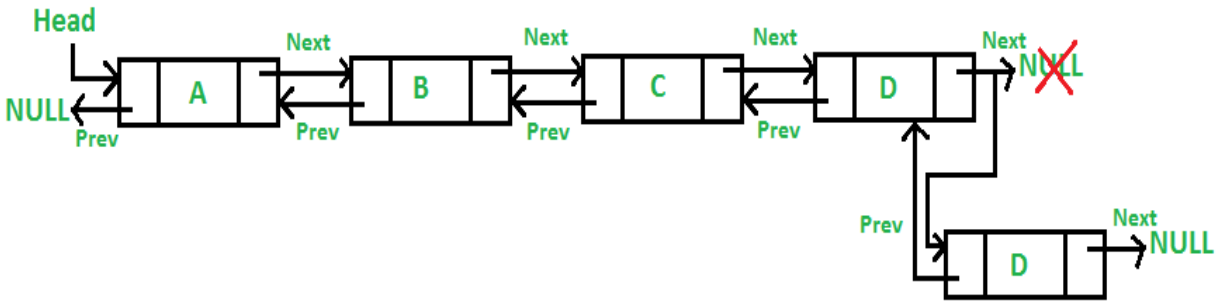
    last = last.next
```

5. **Change the next of last node to newnode.**

```
last.next = new_node
```

6. **Change prev of last node as last**

```
newnode.prev = last
```



Python Code:

```
#Function to add node at the end of linked list
def atEnd(self, data):
    newnode = Node(data)
    if self.head == None:
        self.head = newnode
        return
    last = self.head
    while last.next != None:
        last = last.next

    last.next = newnode
    newnode.prev = last
    return
```

Traversing (print) the Nodes in Doubly linked list

a) Forward Direction

1. Initialize a variable with **head**.
2. Loop that iterates until the end of the doubly linked list is reached.
    - Print the node data.
    - Move the next pointer

b) Reverse Direction

1. Initialize a variable with last node.
2. Loop that iterates until the beginning of the doubly linked list is reached.

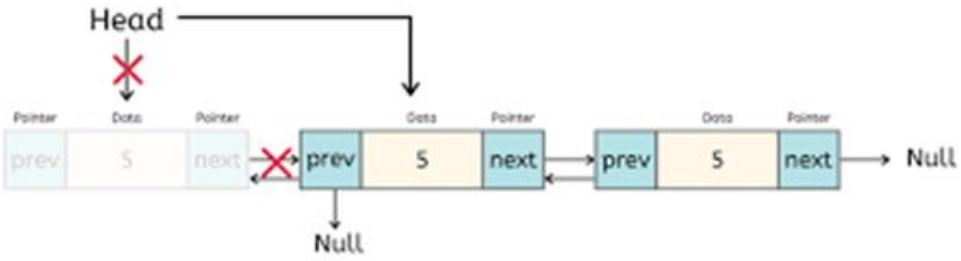
    - Print the node data.
    - Move to the **prev** pointer

```
def printList(self):
      temp = self.head
      print("\nTraversal in forward direction")
      while temp != None:
          print(temp.data, end="  ")
          last = temp
          temp = temp.next

      print("\nTraversal in reverse direction")
      while last != None:
          print(last.data, end="  ")
          last = last.prev
```

Deleting a node at the beginning in DLL

1. Initialize a variable with **head**
2. If linked list is not empty, point head to the second node
3. **Prev** of head node to None



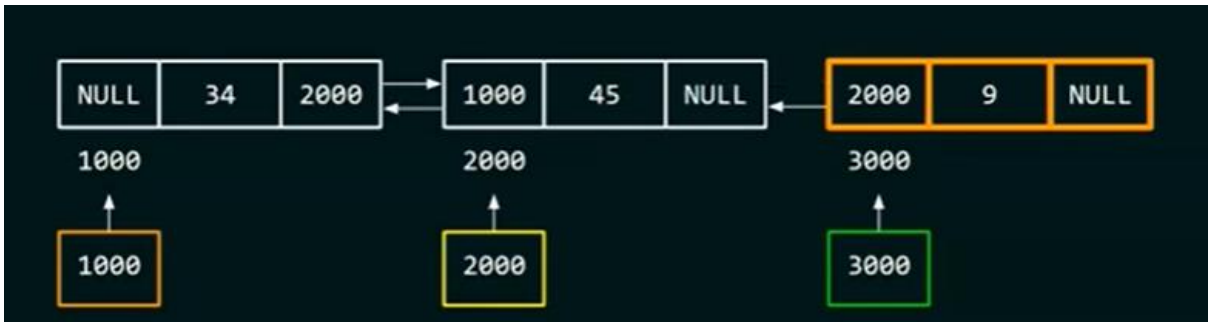
Python code

```
def deleteatbeg(self):
    temp = self.head

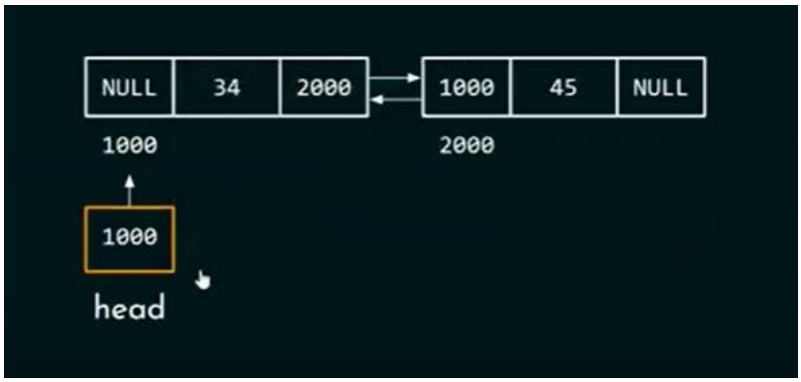
    # if dll is not empty
    if temp != None:
        self.head = temp.next
        self.head.prev = None
```

Deleting a node at the end of DLL

1. If linked list is empty, return
2. if linked list is not empty, Traverse to the last node.
3. Change the **next** of second last (last2) to None.
4. Change the **prev** of last node to None.



Head **last2** **last**



Python code

```
def deleteatend(self):
    if self.head == None:
        return
    else:
        temp = self.head
        while temp !=None:
            last = temp
            temp = temp.next
        last2 = last.prev
        last2.next = None
        last.prev = None
```

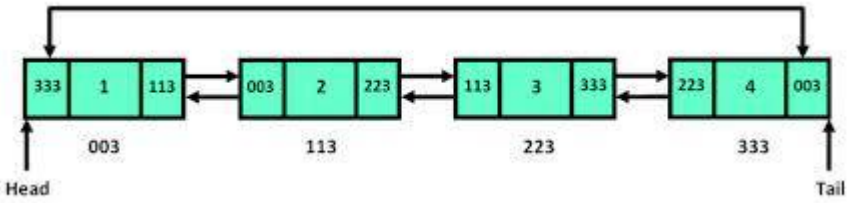
Time Complexity of Linked List

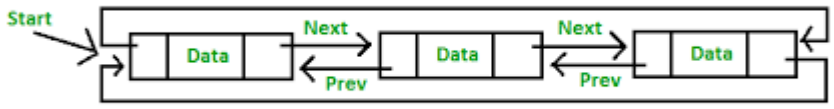
Doubly Linked List	
Insert at beginning	O(1)
Insert at end	O(n)
Deletion at begining	O(1)
Deletion at end	O(n)
Traverse	O(1)

Circular Linked List

Circular linked list is similar to doubly linked list.

The difference is – circular doubly linked list does not contain None in any of the node. The last node of the list contains the address of the first node. The first node contains the address of the last node in previous pointer.





Implementation Of Doubly Linked List

```
class Node:
      def __init__(self, data):
            self.data = data
            self.next = None
            self.prev = None

class DoublyLinkedList:
      def __init__(self):
            self.head = None

      #Function to add node in the beginning
      def AddNode(self, data):
          newnode = Node(data)
          newnode.next = self.head
          if self.head != None:
              self.head.prev = newnode
          self.head = newnode
```

```
    # Function to delete node at beginning
      def DeleteNode(self):
          temp = self.head

          # if dll is not empty
          if temp != None:
              self.head = temp.next
              self.head.prev = None

      def printList(self):
            temp = self.head
            print("\nTraversal in forward direction")
            while temp != None:
                print(temp.data, end="  ")
                last = temp
                temp = temp.next

            print("\nTraversal in reverse direction")
            while last != None:
                print(last.data, end="  ")
                last = last.prev
```

```
#Main
dll = DoublyLinkedList()
dll.AddNode(10)
dll.AddNode(15)
dll.AddNode(20)
print("Doubly Linked List after adding node")
dll.printList( )
print("\n\nDoubly Linked List after deleting node")
dll.DeleteNode()
dll.printList()
```

Output:

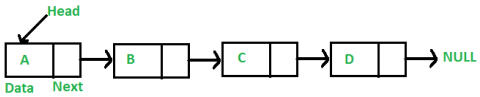
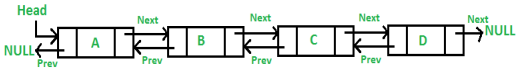
```
Doubly Linked List after adding node

Traversal in forward direction
20  15  10
Traversal in reverse direction
10  15  20

Doubly Linked List after deleting node

Traversal in forward direction
15  10
Traversal in reverse direction
10  15
```

Difference between singly linked list and doubly linked list

Singly linked list (SLL)	Doubly linked list (DLL)
SLL nodes contains 2 field -data field and next link field. 	DLL nodes contains 3 fields - data field, a previous link field and a next link field. 
In SLL, the traversal is done using the next node link only. Thus, traversal is possible in one direction only.	In DLL, the traversal is done using the previous node link or the next node link. Thus, traversal is possible in both directions (forward and backward).
The SLL occupies less memory than DLL as it has only 2 fields.	The DLL occupies more memory than SLL as it has 3 fields.
Complexity of insertion and deletion at a given position is O(n).	Complexity of insertion and deletion at a given position is O(n / 2) = O(n) because traversal can be made from beginning or from the end.
Singly linked list is used to implementation of stacks.	Doubly linked list is used to implement heaps stacks and binary trees.

Difference between recursion and iteration.

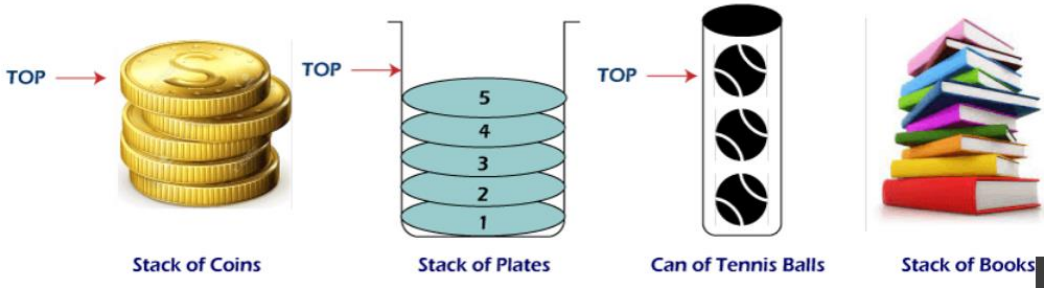
	Recursion	Iteration
1.	Recursion is the process of function calling itself.	In iteration, loops are used to execute set of instructions repeatedly executed.
2.	Recursion is applied for functions.	Iteration is used in loops.
3.	Termination condition is defined within the recursive function.	Termination condition is defined in the definition of the loop.
4.	Code size is smaller	Code Size is larger.
5.	Very high (exponential) time complexity.	Relatively lower time complexity (polynomial-logarithmic).
6.	It has to update and maintain the stack.	There is no utilization of stack.

WEEK – 8 LAST IN FIRST OUT DATA STRUCTURES - STACK

A stack is a linear data structure that uses Last-In First-Out (LIFO) or First-In Last-Out method to store data. It has one end called **top**. Data are inserted and deleted from the top of the stack.

Stack is an abstract data type with pre-defined capacity, which means that it stores the elements of limited size.

An everyday analogy of a stack data structure is a stack of books on a desk, Stack of plates, table tennis, Stack of bootless, Undo or Redo mechanism in the Text Editors, etc



.

Stack operations:

The following are the common operations implemented on the stack –

1. **Create stack** – used to create the stack
2. **Push** – used to insert new element into a stack
3. **Pop** – used for the removal of a element from a stack
4. **Peek** – used to retrieve the element at the top of stack without deleting it.
5. **isEmpty** – used to check whether stack is empty or not
6. **isFull -** used to check whether stack is full or not
7. **deletestack** – used to remove all the data element and free up the allocated memory space.

Stack Implementation:

Stack in Python can be implemented using the following ways:

- list
- Collections.deque
- queue.LifoQueue
- Linked list

The advantage of using list data structure in python is easy to implement. The built-in functions of list such as append( ) and pop( ) is used to insert and delete data from stack.

But the Stack implementation using lists becomes slower as the size of the list increases.

Python stack can be implemented using the deque class from the collections module. Deque is preferred over the list in the cases where quicker append and pop operations are needed.

Queue module also has a LIFO Queue, which is basically a Stack. Data is inserted into Queue using the put( ) function and get( ) takes data out from the Queue.

Creating a stack using list with size limit

```
class stack:
    def __init__(self,max,top):
        self.maxsize = max
        self.top = top
        self.list = []
```

Time and space complexity

The time complexity of creating a stack using list is O(1) as it takes constant amount of time to initialize a list.

The space complexity is O(1) since no additional memory is required.

Push Operation

The push operation is used to insert elements into a stack. The built-in function “append" is used to enter elements in the list. Stack has limited size. The process of adding elements is stopped when stack is full.

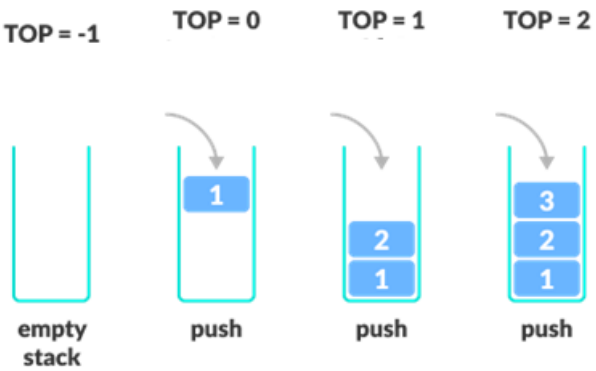
Stack overflow occurs when trying to push elements into stack, when stack is full.

The steps involved in the push operation are –

1. Checks if the stack is full.
2. If the stack is full, display stack is full.
3. If the stack is not full, increments **top** by 1 to point next memory space.
4. Adds data element to the stack, where top is pointing.

Example: Consider,

maxsize of stack is 3.



Python Code:

```
def push(self,data):
    if self.top == self.maxsize -1 :
        print("Stack is full")
    else:
        self.top = self.top + 1
        self.list.append(data)
        print(data, "has been added to stack")
```

Time and space complexity

The time complexity to push an element into a stack using list is O(1) or $O(n^2)$.

The space complexity is O(1) since no additional memory is required.

Pop Operation

Pop operation is used to delete elements from a stack. The built-in function "pop" is used delete elements from the stack.

Stack underflow occurs, when try to delete the element from the empty stack.

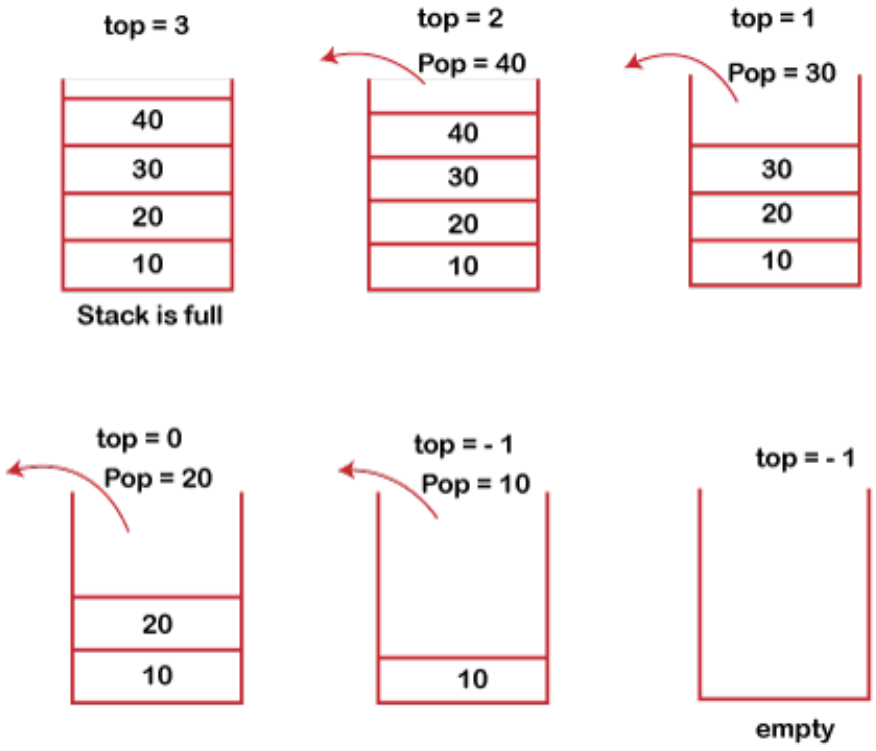
The steps involved in the pop operation are –

1. Checks if the stack is empty.
2. If the stack is empty, then display stack is empty.
3. If the stack is not empty, remove the element from the stack which is pointed by the top.
4. Top is decremented by 1.

Python Code:

```
def pop(self):
    if self.top == -1 :
        print("Stack is empty")
    else:
        print("Popped item = ", self.list.pop())
        self.top = self.top - 1
```

Example: Consider the maxsize of stack is 4.



Time and space complexity

The time complexity to pop an element from a stack using list is O(1) as it takes constant amount of time.

The space complexity is O(1) since no additional memory is required.

Application of stack

1. Used for expression evaluation
2. Conversion of one form of expression to another
3. Reversing input data
4. Solving the parenthesis problem
5. Backtracking problem

Implement Stack Data Structure

```
class stack:
    def __init__(self,max,top):
        self.maxsize = max
        self.top = top
        self.list = []

    def push(self,data):
        if self.top == self.maxsize -1 :
            print("Stack is full")
        else:
            self.top = self.top + 1
            self.list.append(data)
            print(data, "has been added to stack")

    def pop(self):
        if self.top == -1 :
            print("Stack is empty")
        else:
            print("Popped item = ", self.list.pop())
            self.top = self.top - 1
```

```
    def display(self):
        if self.top == -1 :
            print("Stack is empty")
        else:
            print("contents of stack are:")
            for i in reversed(self.list):
                print(i)

# Main
s = stack(3, -1)
s.push(10)
s.push(20)
s.push(30)
s.push(40)

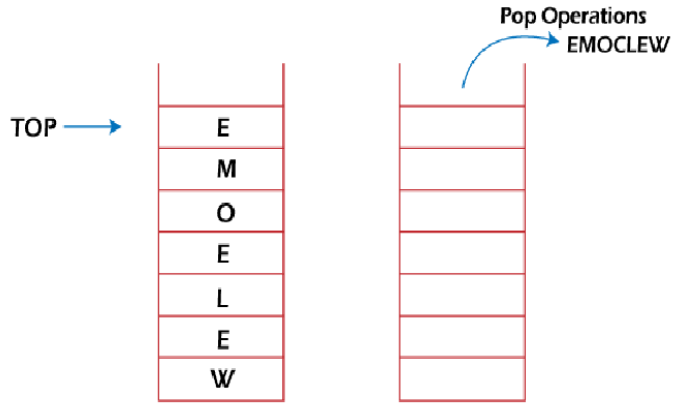
s.display()
s.pop()
s.display()
s.pop()
s.display()
s.pop()
s.display()
```

Output:

```
10 has been added to stack
20 has been added to stack
30 has been added to stack
Stack is full
contents of stack are:
30
20
10
Popped item =  30
contents of stack are:
20
10
Popped item =  20
contents of stack are:
10
Popped item =  10
Stack is empty
```

Example: Reversing a word

A word consists of multiple characters. A Stack can be used to reverse the characters of a word. This can be done by simply pushing one by one each character onto the Stack, which later can be popped from the Stack one by one. Because of the **Last-in First-out** property of the Stack, the first character of the string is on the bottom of the Stack and the last character of the String is on the Top of the Stack and after performing the pop operation in the Stack, the Stack returns the word in Reverse order.



Evaluating an expression

A stack is a very effective data structure for evaluating arithmetic expressions in programming languages. An arithmetic expression consists of operands and operators.

In addition to operands and operators, the arithmetic expression may also include parenthesis like "left parenthesis" and "right parenthesis".

Example: A + (B - C)

Evaluation of Arithmetic Expression requires two steps:

- First, convert the given expression into special notation.
- Evaluate the expression in this new notation.

Notations for Arithmetic Expression

There are three notations to represent an arithmetic expression:

- Infix Notation
- Prefix Notation
- Postfix Notation

Infix Notation

In infix notation operator is placed between the operands. Infix expressions can be parenthesized or un parenthesized depending upon the problem requirement.
Example: A + B, (C - D) etc.

The above expressions is in infix notation because the operator comes between the operands.

Prefix Notation

In prefix notation, operator is placed before the operands. This notation was introduced by the Polish mathematician and hence often referred to as polish notation.

Example: + A B, - C D etc.

The above expression is in prefix notation because the operator comes before the operands.

Postfix Notation

In postfix notation, operator is places the after the operands. This notation is just the reverse of Polish notation and also known as Reverse Polish notation.
Example: AB +, CD +, etc.

The above expression is in postfix notation because the operator comes after the operands.

Conversion of Arithmetic Expression into various Notations:

Infix Notation	Prefix Notation	Postfix Notation
A * B	* A B	AB*
(A+B)/C	/+ ABC	AB+C/
(A*B) + (D-C)	+*AB - DC	AB*DC-+

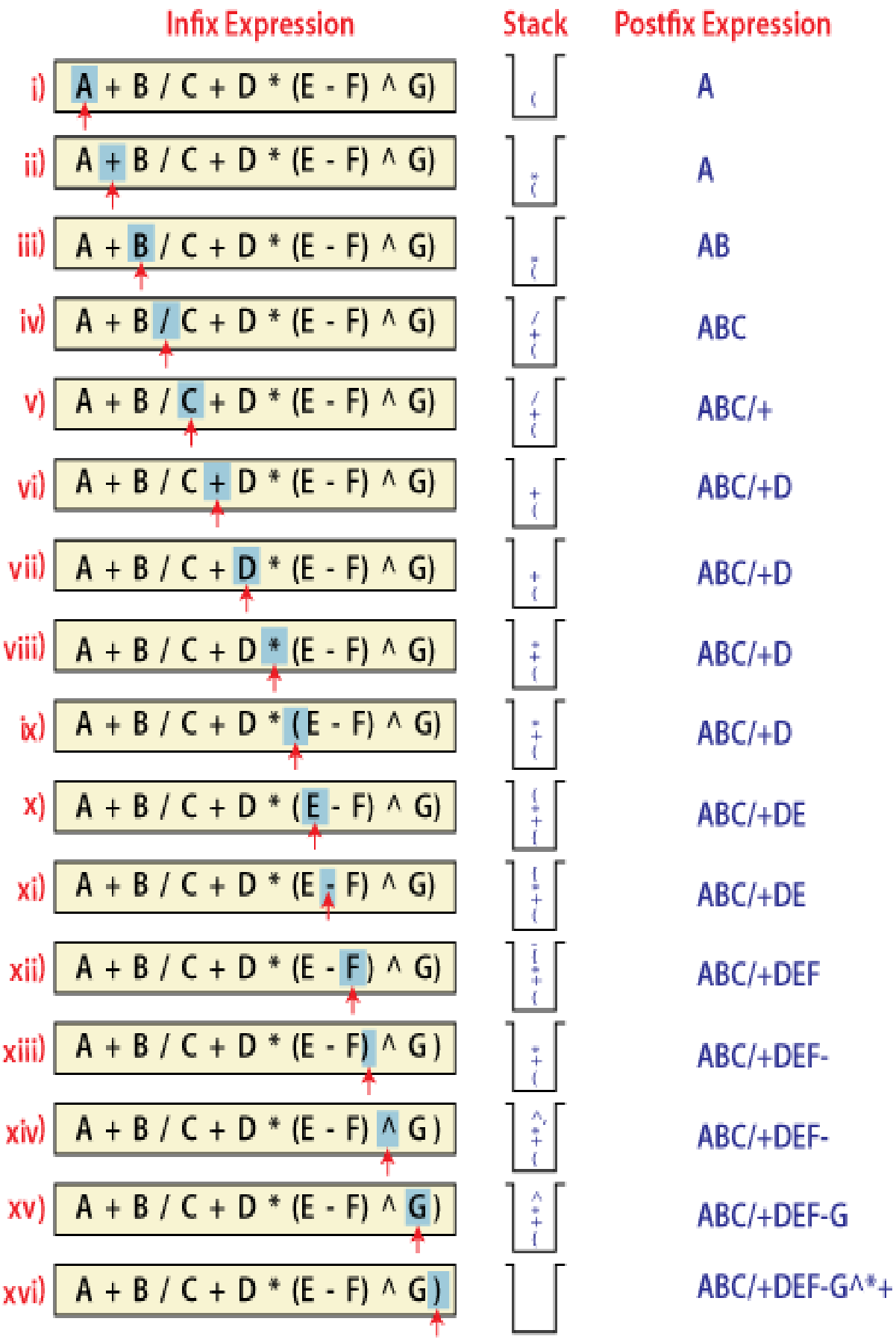
Example of Converting an infix expression into a postfix expression.

The following rules are used:

1. Priority rules for arithmetic operators

Operators	Associativity	Precedence
^ exponentiation	Right to left	Highest followed by *Multiplication and /division
*Multiplication, /division	Left to right	Highest followed by + addition and - subtraction
+ addition, - subtraction	Left to right	lowest

2. No two operators of same priority can stay together in stack
3. Lowest priority cannot be placed on highest priority.


Infix Expression
Stack
Postfix Expression
i) A + B / C + D * (E - F) ^ G)
A
ii) A + B / C + D * (E - F) ^ G)
A
iii) A + B / C + D * (E - F) ^ G)
AB
iv) A + B / C + D * (E - F) ^ G)
ABC
v) A + B / C + D * (E - F) ^ G)
ABC/+
vi) A + B / C + D * (E - F) ^ G)
ABC/+D
vii) A + B / C + D * (E - F) ^ G)
ABC/+D
viii) A + B / C + D * (E - F) ^ G)
ABC/+D
ix) A + B / C + D * (E - F) ^ G)
ABC/+D
x) A + B / C + D * (E - F) ^ G)
ABC/+DE
xi) A + B / C + D * (E - F) ^ G)
ABC/+DE
xii) A + B / C + D * (E - F) ^ G)
ABC/+DEF
xiii) A + B / C + D * (E - F) ^ G)
ABC/+DEF-
xiv) A + B / C + D * (E - F) ^ G)
ABC/+DEF-
xv) A + B / C + D * (E - F) ^ G)
ABC/+DEF-G
xvi) A + B / C + D * (E - F) ^ G)
ABC/+DEF-G^*+

In the above example, the only change from the postfix expression is that the operator is placed before the operands rather than between the operands.

Evaluating Postfix expression:

Stack is the ideal data structure to evaluate the postfix expression because the top element is always the most recent operand. The next element on the Stack is the second most recent operand to be operated on.

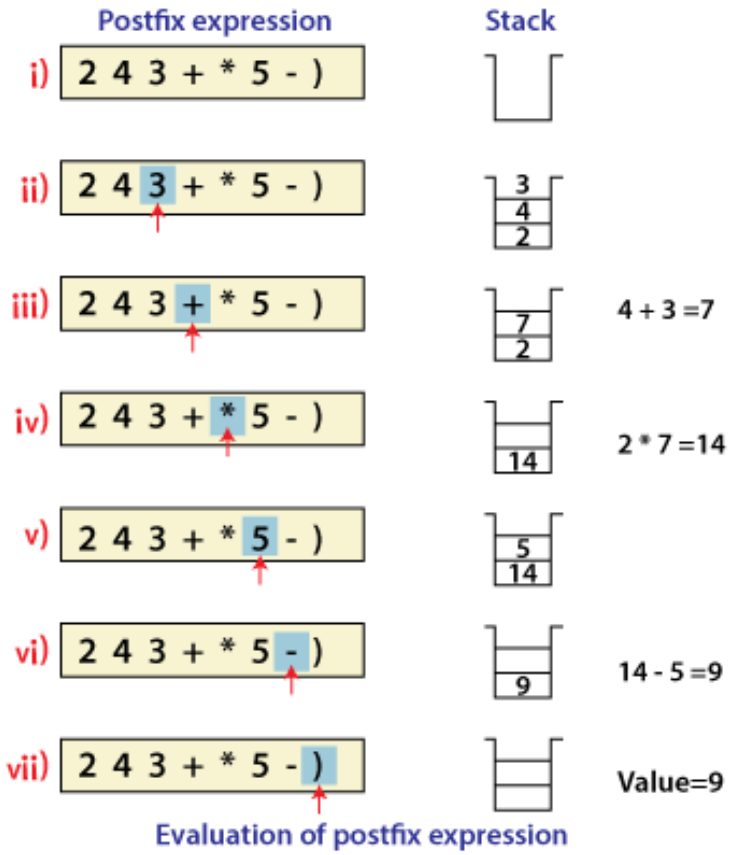
Before evaluating the postfix expression, the following conditions must be checked. If any one of the conditions fails, the postfix expression is invalid.

- When an operator encounters the scanning process, the Stack must contain a pair of operands or intermediate results previously calculated.
- When an expression has been completely evaluated, the Stack must contain exactly one value.

Example: Consider the following infix expression 2 * (4+3) - 5.

Its equivalent postfix expression is 2 4 3 + * 5-.

The following step illustrates how this postfix expression is evaluated.



Message Box

Text editor usually provide an undo option that cancels recent editing operations. This undo operation is done by keeping text changes in a stack. The message i.e., one by one character gets inserted into the stack as and when message is typed. If user wants to undo the change or erase a recent typed character which is done by popping out of stack.

Pop( ) is used to remove or undo the last changes made in message box.

Stack application – Balanced Delimiters

The common application of Stack is delimiter checking, i.e., **parsing that involves analysing a source program syntactically. It is also called parenthesis checking.**

When the compiler translates a source program written in some programming language such as C, C++ to a machine language, it parses the program into multiple individual parts such as variable names, keywords, etc. By scanning from left to right. The main problem encountered while translating is the unmatched delimiters. Different types of delimiters used include the parenthesis checking (,), curly braces {,} and square brackets [,], and common delimiters /* and */. Every opening delimiter must match a closing delimiter, i.e., every opening parenthesis should be followed by a matching closing parenthesis. Also, the delimiter can be nested. The opening delimiter that occurs later in the source program should be closed before those occurring earlier.

Valid Delimiter	Invalid Delimiter
While (i > 0)	While (i >
/* Data Structure */	/* Data Structure
{ (a + b) - c }	{ (a + b) - c

To perform a delimiter checking, the compiler makes use of a stack. When a compiler translates a source program, it reads the characters one at a time, and if it finds an opening delimiter it places it on a stack. When a closing delimiter is found, it pops up the opening delimiter from the top of the Stack and matches it with the closing delimiter.

On matching, the following cases may arise.

- If the delimiters are of the same type, then the match is considered successful, and the process continues.
- If the delimiters are not of the same type, then the syntax error is reported.

When the end of the program is reached, and the Stack is empty, then the processing of the source program stops.

Example: consider the following expression.

[{ a – b) * (c – d) } / f]

Input Left	Character Read	Stack
[{ (a – b) * (c – d) } / f]	[	[
{ (a – b) * (c – d) } / f]	{	[{
(a – b) * (c – d) } / f]	(	[{ (
a – b) * (c – d) } / f]	a	[{ (
– b) * (c – d) } / f]	-	[{ (
b) * (c – d) } / f]	b	[{ (
) * (c – d) } / f]	)	[{
* (c – d) } / f]	*	[{
(c – d) } / f]	(	[{ (
c – d) } / f]	c	[{ (
– d) } / f]	-	[{ (
d) } / f]	d	[{ (
) } / f]	)	[{
} / f]	}	[
/ f]	/	[
f]	f	[
]	]	

Python Code

```
openlist = [ "[", "{", "(" ]
closelist = [ "]", "}", ")" ]

def CheckBalance(mystring):
    stack = []
    for i in mystring:
        if i in openlist:
            stack.append(i)
        elif i in closelist:
            pos = closelist.index(i)
            if (len(stack) > 0 and openlist[pos] == stack[len(stack)-1]):
                stack.pop()
            else:
                return "Unbalanced"
    if len(stack) == 0:
        return "Balanced"
    else:
        return "Unbalanced"

# Main
string = "{ []{()}}"
print(string, '-',CheckBalance(string))

string = "{ [](({"
print(string, '-',CheckBalance(string))

string = "{ [](){"
print(string, '-',CheckBalance(string))
```

Output:

```
{ []{()}} - Balanced
{ [](({ - Unbalanced
{ [](){ - Balanced
```

WEEK – 9 RECURSION

Recursion is the process by which a function or method calls itself directly or indirectly to process its data in a repetitive way.

A function that uses recursion to process its data is called recursive function.

Types of Recursion

1. Direct Recursion – the function calls itself repeatedly until a base condition is satisfied.

```
def  function1( ):
    ……….
    ……….
    Body
    ……….
    function1( )
```

2. Indirect Recursion - when a function calls another function, which in turn calls the first function again, it is called an indirect recursion.

```
# the first function
def  function1( ):
    ……….
    ……….
    Body
    ……….
    function2( )


# the second function
def  function2( ):
    ……….
    ……….
    Body
    ……….
    function1( )
```

Recursion uses stacks to store data of calling function when program control moves to the called function. The inserted data is poped when the control returns to the calling function after executing the called function.

Advantages of Recursion

- Code is easier to implement.
- Reduces length of code.
- Used to implement infix, postfix and prefix expressions.
- Improves time complexity.

Disadvantages of Recursion

- Slower than non-recursive functions.
- The system may go into an infinite loop if the base condition is unchecked.
- Recursive calls are expensive (inefficient) as they take up a lot of memory and time.
- Recursive functions are difficult to debug.

Applications of Recursion

- Used in Depth First Search
- Used in Breadth First Search
- Used in implementing Quick Sort Algorithm, Tower of Hanoi and Fibonacci Series algorithm.

Properties of Recursion

All recursive solutions must satisfy three rules or properties:

1. A recursive solution must contain a **base case**.
2. A recursive solution must contain a **recursive case.**
3. A recursive solution must **make progress toward the base case**.

A recursive solution subdivides a problem into smaller versions of itself. For a problem to be subdivided, it must consist of a data that can be divided into smaller data. This subdivision is handled by the recursive case when the function calls itself.

The **base case** is the terminating case and represents the smallest subdivision of the problem. It signals the end of the loop or recursive calls.

Finally, a **recursive solution must make progress toward the base case** otherwise, the recursion will never stop resulting in an infinite loop. This progression typically occurs in each recursive call when the larger problem is divided into smaller parts. The larger data is subdivided into smaller sets or the larger term is reduced to a smaller value by each recursive call.

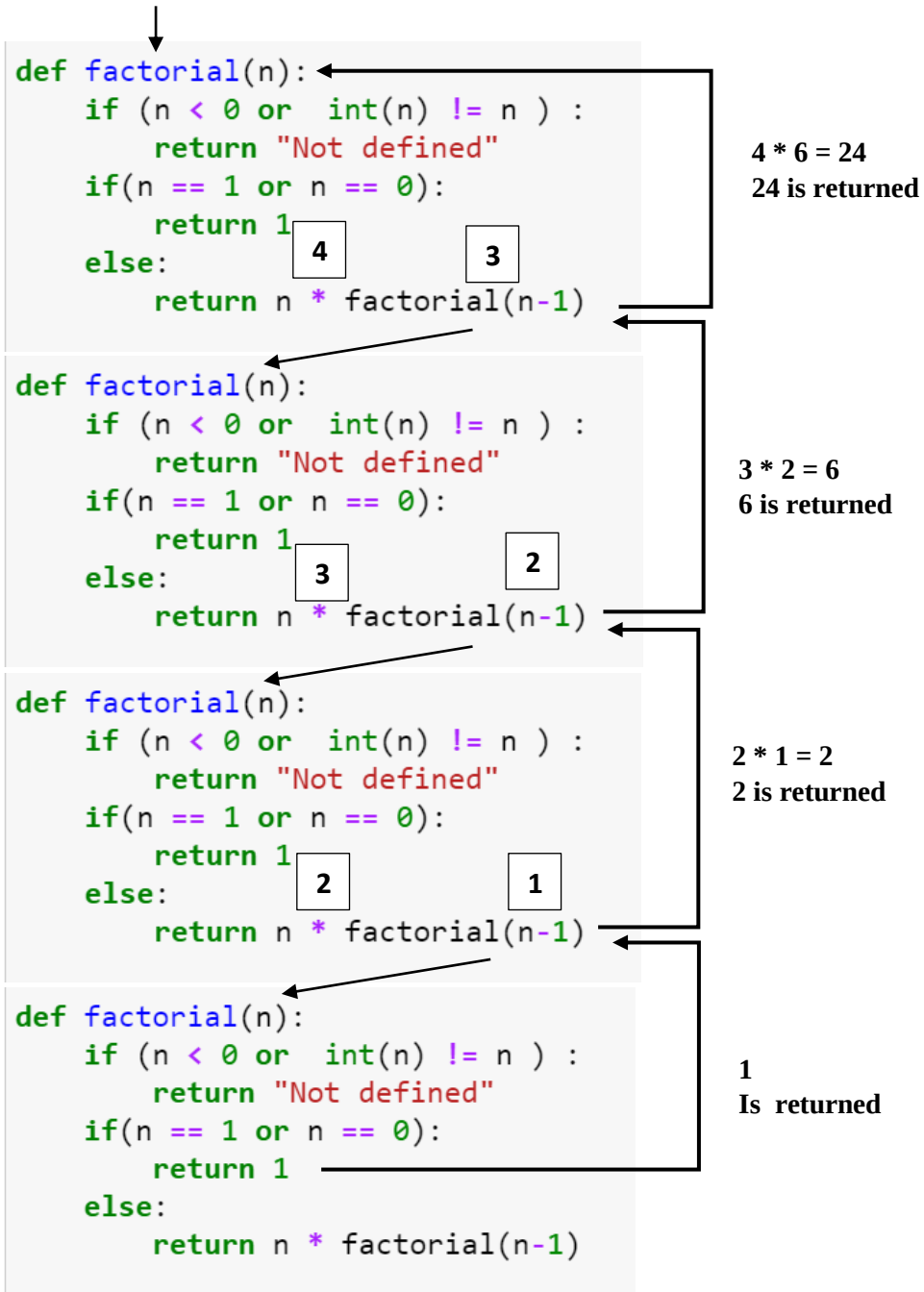
Recursive Function : Factorial

Factorial of a number n (n!) is the product of all integers from 1 to that number. The factorial of numbers 1 and 0 is 1 and the factorial for negative numbers are not defined.

n! = n * (n – 1) * (n – 2) * …….3 * 2 * 1
Mathematically,
Factorial(n) = n * (n – 1) * (n – 2) * …….. 3 * 2 * 1 --------Equation 1
Factorial(n - 1) = (n – 1) * (n – 2) * …….. 3 * 2 * 1 --------Equation 2

From equation 1 and 2 , **Factorial(n) = n * Factorial(n – 1)**

Factorial (4)



Base Case: The smallest possible input for the factorial of a number is 0 and 1. Hence, the recursion stops when the number is reduced to 0 or 1. This is base condition.

Recursive Case: If the number is greater than 1, the function makes recursive call as, **Factorial(n) = n * Factorial(n – 1)**

Unintentional case: The factorial for negative numbers or floating-point numbers are not defined.

Program to demonstrate recursive operations - factorial of a number

```
def factorial(n):
    if (n < 0 or  int(n) != n ) :
        return "Not defined"
    if(n == 1 or n == 0):
        return 1
    else:
        return n * factorial(n-1)

# Main
n = eval(input("Enter the number: "))
print("Factorial of a number : ", factorial(n))
```

Output 1:

```
Enter the number: 5
Factorial of a number :  120
```

Output 2:

```
Enter the number: -12
Factorial of a number :  Not defined
```

Recursive Stack

Recursive function use call stack to keep track of function calls. When a program calls a function, that function is pushed on top of call stack. When return

Example: Factorial(4) 4! = 4 * 3 * 2 * 1

When each time the code calls a function, that function gets added(pushed) to the stack. Once a function ends, it is removed from the stack (poped) and returns to the calling function.

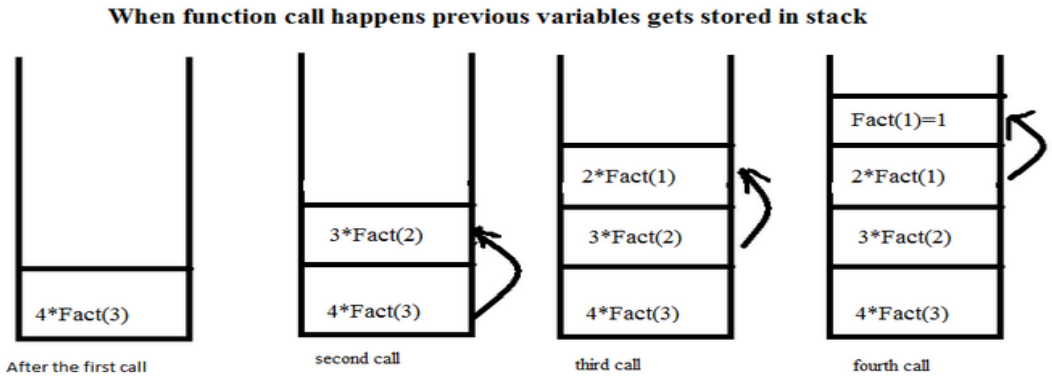
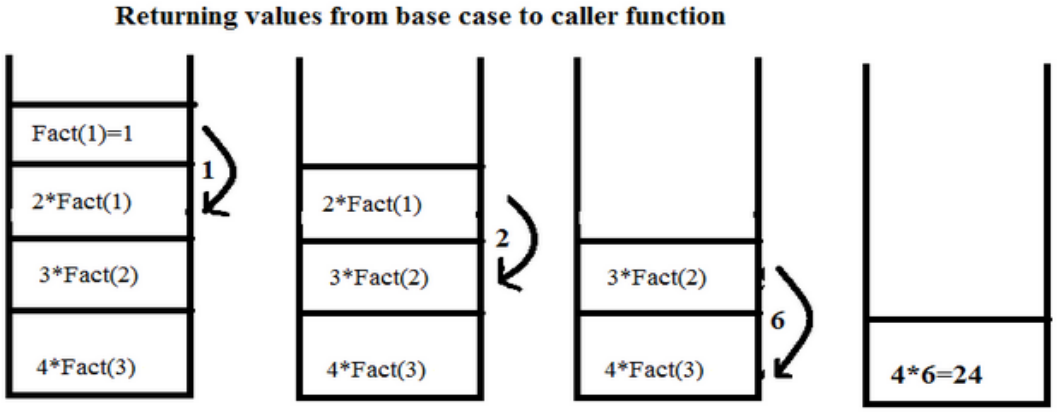


Figure shows call stack having 4 function calls. They stay in stack until the last value is is added. Once last value is obtained, the function calls are poped as shown in below figure.



Fibonacci Series

Fibonacci series is a series of numbers in which each number is the sum of two preceding numbers. The sequence starts from 0 and 1.

The Fibonacci sequence is

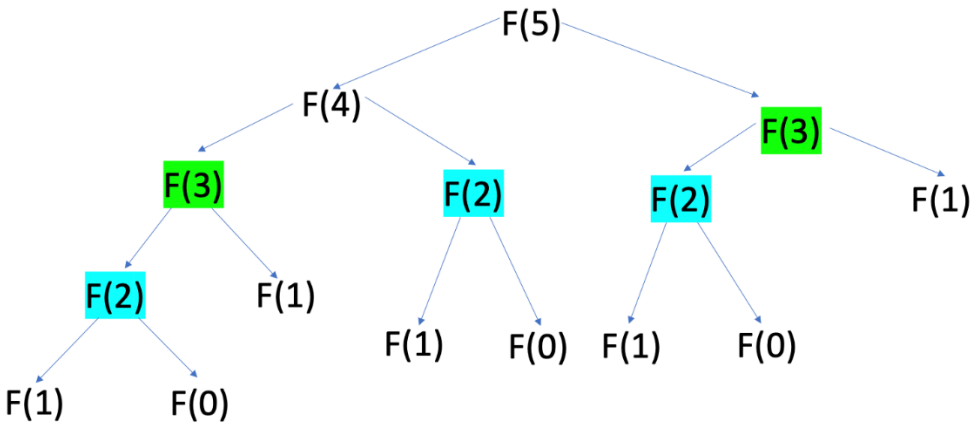
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89 …..

Base Condition: The series starts from 0 and 1. Hence, fib(0) = 0 and fib(1) = 1

Recursive Case: The function makes recursive call as,

fib(n) = fib(n-1) + fib(n-2)

Unintentional cases: The number n, should not be a negative number or floating-point number.



Program to demonstrate recursive operations - Fibonacci series

```
def fib(n):
        if n == 0 or n == 1 :
            return n
        else:
            return fib(n-1) + fib(n-2)

# Main
n = eval(input("Enter the number : "))
if (n < 0 or int(n) != n):
    print("Please enter positive integer")
else:
    print("Fibonacci Series :")
    for i in range(0, n):
        print(fib(i), end ='   ')
```

Output 1:

```
Enter the number : 9
Fibonacci Series :
0   1   1   2   3   5   8   13   21
```

Output 2:

```
Enter the number : -12
Please enter positive integer
```

Output 3:

```
Enter the number : 5.34
Please enter positive integer
```

How Recursion Works

When a function is called, the sequential flow of execution is interrupted and execution jumps to the body of that function. When the function terminates, execution returns to the point where it was interrupted before the function was invoked.

The Run Time Stack

Each time a function is called, an **activation record** is automatically created in order to maintain information related to the function. Activation record contains information about return address. This is the location of the next instruction to be executed when the function terminates. Thus, when a function returns, the address is obtained from the activation record and the flow execution can return to where it left off before the function was called. The activation records also include storage space for the allocation of local variables.

An activation record is created per function call. When a function is called, an activation record is created for that call and when it terminates the activation record is destroyed. The system must manage the collection of activation records and remember the order in which they were created. It does this by storing the activation records on a **run time stack.** The run time stack is just like the stack. but it's hidden from the programmer and is automatically maintained.

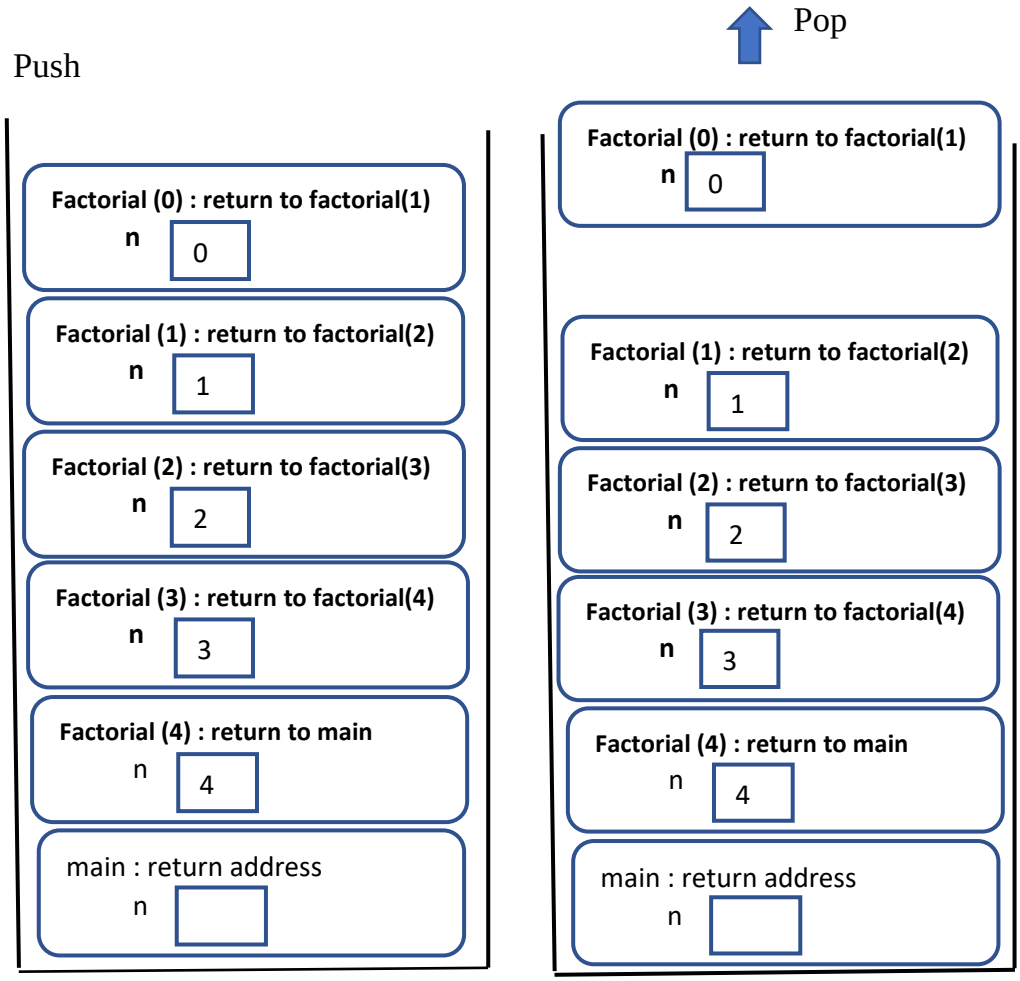
Consider the execution of the following code segment,

```
def factorial(n):
    if (n < 0 or  int(n) != n ) :
        return "Not defined"
    if(n == 1 or n == 0):
        return 1
    else:
        return n * factorial(n-1)

# Main
n = eval(input("Enter the number: "))
print("Factorial of a number = ", factorial(n))
```

When the factorial function is call main routine is executed, the first activation record is created and pushed onto the run time stack, as shown in Figure.

The factorial function is recursively called until the base case is reached with a value of n = 0. At this point, the run time stack contains four activation records. When the base case statement is executed, the activation record for the function call fact(0) is popped from the stack, and execution returns to the function instance fact(1). This process continues until all of the activation records have been popped from the stack and the program terminates.



Recursive Applications- Recursive Binary Search

Binary search is a searching algorithm which is used to search an element from a **sorted** list. Binary search is an efficient algorithm and is better than linear search in terms of time complexity. Binary Search uses divide and conquer technique

It works as follows

1. It compares a search key *with* the list's middle element. If they match, then index of matching element is returned and stops.
2. If search key is lesser than middle element, then key is searched in sub list to the left of middle element.

3. If search key is greater than middle element then key is searched in the sub list to the right of the middle element.
4. This process continues on the sub list as well until the size of the sub list reduces to zero.

Python code for Binary Search (Recursive Method)

```
def BinarySearch(mylist, key, low, high):
    if low <= high :

        mid = (low + high) // 2

        if key == mylist[mid] :
            return mid

        elif key < mylist[mid] :
            return BinarySearch(mylist, key, low, mid-1)

        else:
            return BinarySearch(mylist, key, mid + 1, high)

    else:
        return -1

#Main
mylist = [11, 12,13, 14, 15, 16, 17, 18]
key = int(input("Enter key  "))
n = len(mylist)
pos = BinarySearch(mylist,key, 0, n-1)
if pos == -1:
    print("UnSuccessfull search")
else:
    print("Sucessfull search: Element found at : ",pos)
```

Output:

```
Enter key  13
Sucessfull search: Element found at :  2
```

Towers of Hanoi

The "**Tower of Hanoi"** puzzle is - There are three **rods**. The disks are stacked in the descending order. The largest disk stacked at the bottom and the smallest one on top. The task is to transfer the disks from one source rod to another target rod.

The following rule must be followed -

- Only one disk can be moved at a time.
- The uppermost disk from one of the rod can be moved.
- The larger disk cannot be placed on the smaller disk.

The number of moves can be calculated as $\mathbf{2^n - 1}$.

If there are three disks on the first rod, then total 7 moves are needed.

The most left rod is called **SOURCE**, and the rightmost rod is called **TARGET**. The middle rod is referred to as an **AUX**.

The AUX is needed to deposit disks temporarily.

Algorithm:

1. Create a **tower_of_hanoi** recursive function and pass two arguments: the number of disks n and the name of the rods such as **source, aux,** and **target**.
2. Define the base case- If the number of disks is 1. Then simply move the one disk from the **source** to **target** and return.
3. Now, move remaining n-1 disks from **source** to **aux** using the target as the auxiliary.
4. Then, the remaining 1 disk move on the **source** to **target**.
5. Move the n-1 disks on the **aux** to the **target** using the **source** as the auxiliary.

Implement solution for Towers of Hanoi.

```
def towerofhanoi(disk, source, aux, target):
    if(disk == 1):
        print("Move disk 1 from rod {} to rod {}".format(source,target))
        return

    towerofhanoi(disk-1, source, target, aux)

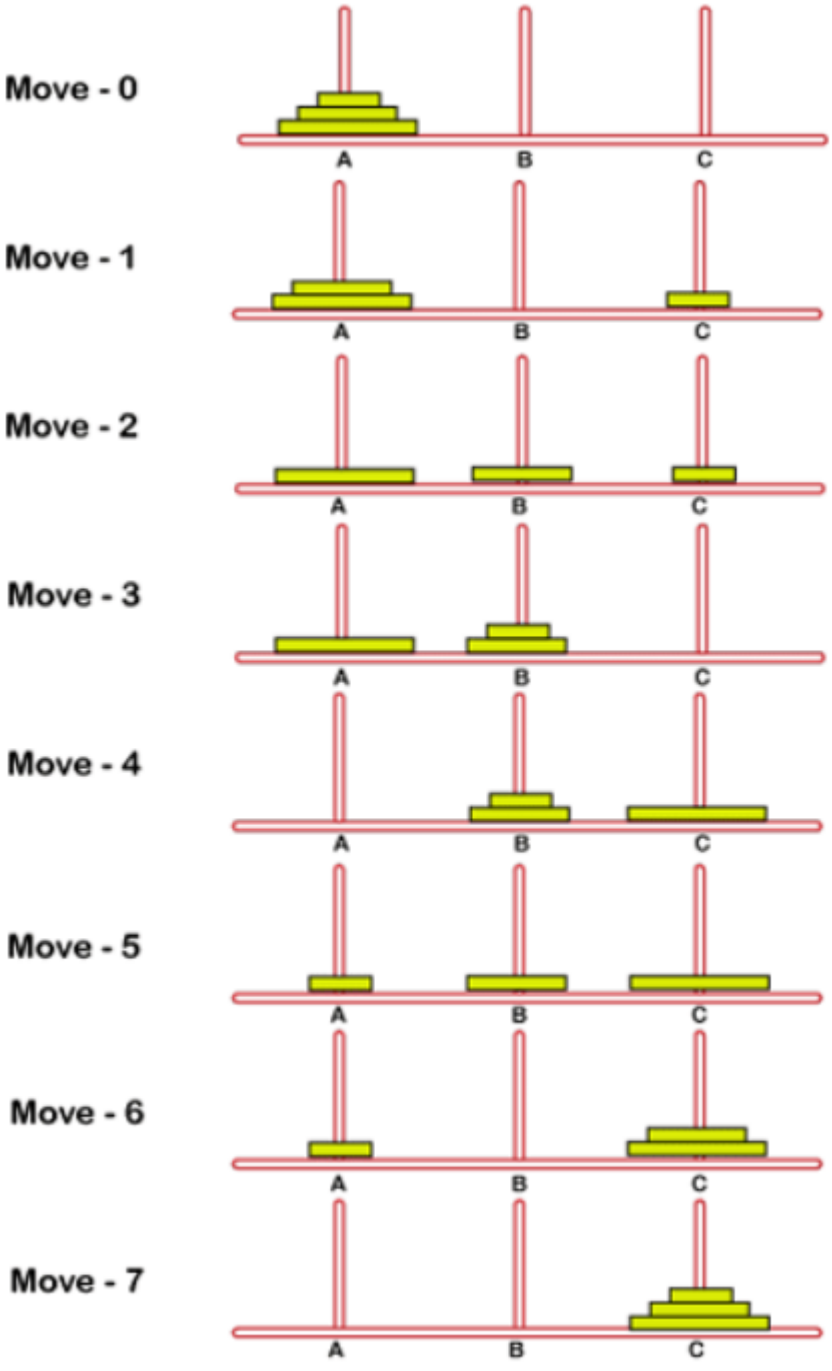
    print("Move disk {} from rod {} to rod {}".format(disk,source,target))

    towerofhanoi(disk-1, aux, source, target)

# Main
disk = int(input("Enter the number of disk: "))
towerofhanoi(disk, 'A','B','C')
```

Output:

```
Enter the number of disk: 3
Move disk 1 from rod A to rod C
Move disk 2 from rod A to rod B
Move disk 1 from rod C to rod B
Move disk 3 from rod A to rod C
Move disk 1 from rod B to rod A
Move disk 2 from rod B to rod C
Move disk 1 from rod A to rod C
```



Tower of Hanoi